MOBL-03: Android SDK Setup (Kotlin & Jetpack Compose)

> **Series:** MOBL — Mobile Monitoring | **Notebook:** 3 of 12 | **Created:** February 2026 | **Last Updated:** 11/11/2026

Overview

This notebook walks through setting up Dynatrace Mobile RUM (Real User Monitoring) for Android applications using the Gradle plugin. It covers both traditional View-based architectures and modern Jetpack Compose UIs, including auto-instrumentation, manual action tracking, and data verification via DQL.

Table of Contents

1. Creating a Mobile App in Dynatrace
2. Gradle Plugin Setup
3. build.gradle Configuration
4. Auto-Instrumentation (Activities & Fragments)
5. Jetpack Compose Integration
6. ProGuard & R8 Configuration
7. Verifying Data in Dynatrace

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail, with a mobile frontend created in Experience Vitals (RUM Classic frontends: RUM turned on for the frontend; Section 1)
Minimum versions	Android API level 23, Gradle 8.0, Android Gradle Plugin 8.1.1, Java 17, Kotlin 2.1.0 (an Android Studio release that supports AGP 8.1.1+); Kotlin Gradle plugin 2.2 from OneAgent for Mobile 8.351 (note below)
Jetpack Compose	1.4 – 1.10; OneAgent for Mobile 8.351 adds 1.12 (note below)
Permissions	Dynatrace admin access to create mobile applications; `storage:user.events:read` and `storage:smartscape:read` for the verification queries
Prior Knowledge	MOBL-01 and MOBL-02 recommended

> **OneAgent for Mobile 8.351 (published 10/27/2026, staged rollout from 11/03/2026 — check which agent version your build pulls in).** Breaking change: *"OneAgent for Android requires the Kotlin Gradle plugin minimum version 2.2. Projects using older Kotlin compiler versions must update their Kotlin Gradle plugin to 2.2 or higher."* The same release adds that the Dynatrace Android Gradle plugin *"now supports auto-instrumentation for Jetpack Compose version 1.12"*. The support page still lists Kotlin 2.1.0 and Compose 1.4 – 1.10 (read 11/11/2026); until you move to 8.351, those remain the minimums for the agent version you build with.
>
> **Sources:** Support and limitations — Android (DT docs) — *"Android API level 23 Gradle 8.0 Android Gradle Plugin 8.1.1 Java 17 Kotlin 2.1.0 Jetpack Compose 1.4 - 1.10"* (minimum versions, read 11/11/2026); What's new in OneAgent for Mobile 8.351 (DT docs) — *"OneAgent for Android requires the Kotlin Gradle plugin minimum version 2.2."*, *"The Dynatrace Android Gradle plugin now supports auto-instrumentation for Jetpack Compose version 1.12."*, read 11/11/2026.

1. Creating a Mobile App in Dynatrace

Before instrumenting your Android project, you need to register the application in Dynatrace to obtain the **Application ID** and **Beacon URL**.

Steps

1. **Create the frontend:** open **Experience Vitals**, select **Frontend** in the upper-right corner, select the **Mobile** type, and follow the Frontend creation wizard with **Android** as the platform and a meaningful name (e.g., `My Android App - Production`).
2. From the instrumentation wizard, note:

Property	Description	Example
Application ID	Unique identifier for your mobile app in Dynatrace	`abcd1234-5678-efgh-ijkl-mnopqrstuvwx`
Beacon URL	Endpoint where the agent sends monitoring data	`https://{your-env-id}.bf.dynatrace.com/mbeacon`

> **Tip:** Keep the Application ID and Beacon URL in a secure location. You will paste them into your `build.gradle.kts` in a later step.

Optional Configuration

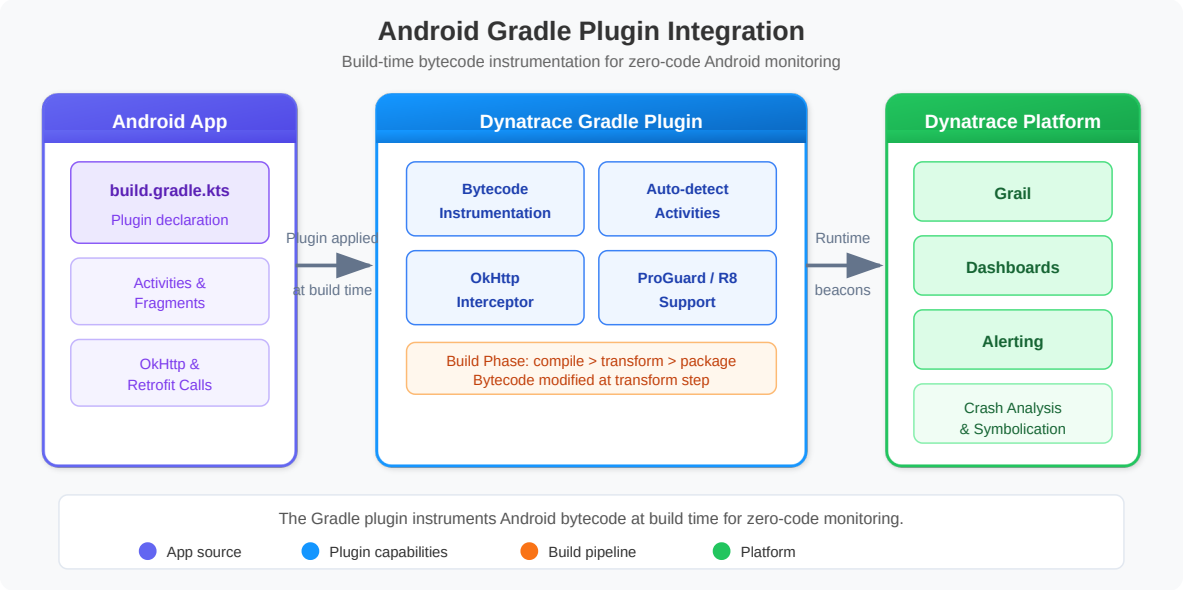
While in the Dynatrace UI, review these settings:

Setting	Recommendation
Crash reporting	Enable (captures unhandled exceptions)
User action naming	Configure meaningful rules for key screens
Session replay	Enable if you need visual session data (increases data volume)
Data privacy	Configure according to your organization's requirements

> **Upgrading a RUM Classic Android frontend?** Turn RUM on for it: **Experience Vitals > Overview > Mobile** → select the frontend → **Settings** → under **Enablement and cost control**, turn on **RUM**. For all mobile frontends at once: **Settings > Collect and capture > Real User Monitoring > Enablement and cost control > Mobile** → **RUM**. The `user.events` / `user.sessions` queries in this series read the data RUM sends to Grail.
>
> **Sources:** Initial setup for Android frontends (DT docs) — *"Select Frontend in the upper-right corner. Select the Mobile type of frontend."*; Enable RUM for Android frontends (DT docs) — *"If you already monitor your Android frontends with RUM Classic, upgrading to RUM requires only a configuration change."*, *"Under Enablement and cost control , turn on RUM ."*

2. Gradle Plugin Setup

The Dynatrace Android Gradle plugin handles bytecode instrumentation at build time. It automatically instruments Activity and Fragment lifecycle events, network requests, and UI interactions without requiring code changes.



Plugin Management (settings.gradle.kts)

Add the Dynatrace plugin repository to your `settings.gradle.kts`:

```
// settings.gradle.kts (Plugin Management)
pluginManagement {
    repositories {
        gradlePluginPortal()
        google()
        mavenCentral()
    }
}
```

Top-Level build.gradle.kts

Apply the Dynatrace plugin in the **top-level** build file (the one in the root project directory), not in the app module. From there the plugin configures the Android subprojects:

```
// build.gradle.kts (top-level, root project directory)
plugins {
    id("com.android.application") version "8.5.0" apply false
    id("com.dynatrace.instrumentation") version "8.+" apply true
}

dynatrace {
    configurations {
        create("sampleConfig") {
            autoStart {
                applicationId("&lt;YourApplicationID&gt;")
                beaconUrl("&lt;ProvidedBeaconURL&gt;")
            }
        }
    }
}
```

> **Important:** Copy the `applicationId` and `beaconUrl` values from the instrumentation wizard of your mobile app in Dynatrace. The docs recommend `8.+` so Gradle picks up new minor versions automatically; upgrade across a major version by hand, because a new major can contain breaking changes. If your build needs reproducible versions, pin an exact `8.x.y` instead and bump it deliberately.
>
> **Correction (09/28/2026).** Earlier revisions applied the plugin in the app-level build file and configured it with a `defaultConfig { applicationId(...) }` block plus `variants { register(...) }` overrides. That DSL does not exist. The documented shape is the

> top-level `dynatrace { configurations { ... } }` block shown here, with the IDs inside `autoStart { }`.
>
> **Sources:** Instrumentation via Dynatrace Android Gradle plugin (DT docs) — *"You should apply the Dynatrace Android Gradle to the top-level build file"*, Configure multi-module projects (DT docs).
>
> **Gradle 9.7 is supported (OneAgent for Mobile 8.345, released 08/03/2026).** The Dynatrace Android Gradle plugin adds *"Support for Gradle version 9.7"*. If your project is pinned below that in `gradle-wrapper.properties`, this removes the plugin as a reason not to move; if you are already on 9.7, use the 8.345 plugin or later. Build 8.345.1 also resolves two ANRs in **Jetpack Compose** instrumentation — one in Compose with Session Replay, one in Compose user-interaction monitoring — worth knowing if Compose instrumentation was deferred on those grounds.
>
> **Sources:** What's new in OneAgent for Mobile 8.345 (DT docs) — Gradle 9.7 support and the Android 8.345.1 fixes, read 08/28/2026.
>
> **Breaking (OneAgent for Mobile 8.349, published 10/15/2026, rollout start 10/06/2026).** *"The debug-only certificate validation option that disabled SSL certificate checks has been removed from the Dynatrace Android Gradle plugin (the certificateValidation property) and the OneAgent for Android SDK (ConfigurationBuilder#withCertificateValidation)." "Builds that still reference it will fail to compile."* Remove `certificateValidation` / `withCertificateValidation` before you upgrade; for trust rules and certificate handling the release note points to Android's Network Security Configuration.
>
> **Sources:** What's new in OneAgent for Mobile 8.349 (DT docs) — *"Remove all references to this option from your Gradle plugin configuration and OneAgent initialization code before upgrading"*, read 10/29/2026.

3. build.gradle Configuration

The plugin is configured through **named, variant-specific configurations**. Each configuration is matched to Android build variants by the regex in `variantFilter`, and the plugin cancels the build if a variant has no matching configuration (switch that protection off with `strictMode`).

A typical setup sends debug builds and release builds to two separate mobile apps in Dynatrace:

```
// build.gradle.kts (top-level)
configure&lt;com.dynatrace.tools.android.dsl.DynatraceExtension&gt; {
    configurations {
        create("debug") {
            variantFilter("[dD]ebug")
            autoStart {
                applicationId("&lt;DebugApplicationID&gt;")
                beaconUrl("&lt;ProvidedBeaconURL&gt;")
            }
        }
        create("prod") {
            variantFilter("[rR]elease")
            autoStart {
                applicationId("&lt;ProductionApplicationID&gt;")
                beaconUrl("&lt;ProvidedBeaconURL&gt;")
            }
        }
    }
}
```

To stop monitoring debug builds entirely, set `enabled(false)` on the debug configuration instead of giving it IDs.

Configuration Properties

Property	Where	Description
`variantFilter`	configuration	Regex matched (case-sensitive) against the build-variant name; the first matching configuration wins
`autoStart { applicationId, beaconUrl }`	configuration	IDs from the instrumentation wizard; OneAgent starts automatically with them
`enabled`	configuration	`false` disables monitoring for the variants the configuration matches

Property	Where	Description
`userOptIn`	configuration	`true` starts OneAgent in user opt-in mode (see MOBL-09 §3)
`crashReporting`	configuration	Crash reporting is **on by default**; `crashReporting(false)` turns it off
`hybridWebView { enabled }`	configuration	Hybrid (WebView) monitoring properties live in the `hybridWebView` block
`strictMode`	extension	Controls whether an unmatched variant fails the build

```
// Opt-in mode, crash reporting off, hybrid WebView monitoring on -- each is a documented sample
configure&lt;com.dynatrace.tools.android.dsl.DynatraceExtension&gt; {
    configurations {
        create("sampleConfig") {
            userOptIn(true)
            crashReporting(false)
            hybridWebView {
                enabled(true)
                domains(".&lt;domain1&gt;", ".&lt;domain2&gt;")
            }
        }
    }
}
```

> **Warning:** Never commit real Application IDs or Beacon URLs to public repositories. Use environment variables or a `local.properties` file excluded from version control.
>
> **Sources:** Configure plugin for instrumentation processes (DT docs), Adjust OneAgent configuration (DT docs) — *"All properties related to hybrid application monitoring are part of HybridWebView DSL, so configure them via the hybridWebView block."*, Configure monitoring capabilities (DT docs) — *"You can deactivate crash reporting with the crashReporting property."*

4. Auto-Instrumentation (Activities & Fragments)

The Dynatrace Gradle plugin automatically instruments the following interactions at build time -- no code changes required:

Automatically Detected

Category	What Is Captured	Notes
Activity lifecycle	`onCreate`, `onResume`, `onPause`, `onDestroy`	Load actions generated per Activity
Fragment lifecycle	Fragment attach, detach, view creation	Visible as child actions
Button clicks	`View.OnClickListener` events	Captured with view ID and text
Jetpack Compose	`Modifier.clickable`, `combinedClickable`, `toggleable`, `swipeable`, `pullRefresh`, `Slider` / `RangeSlider`, `HorizontalPager`	Enabled by default from plugin 8.271
RecyclerView taps	Item click events in lists	Requires standard click listeners
OkHttp requests	HTTP/HTTPS calls via OkHttp 3, 4 and 5	Headers injected for distributed tracing
HttpURLConnection	Standard Java HTTP calls	Auto-correlated to user actions
WebView actions	Page loads and JS interactions	Requires `hybridWebView { enabled(true) }`

How It Works

1. **Build time:** The Gradle plugin applies bytecode instrumentation to compiled classes.
2. **Runtime:** The Dynatrace agent initializes automatically on `Application.onCreate()`.

3. **User actions:** Each Activity load or tap generates a user action with child events (network calls, errors).
4. **Beacons:** Collected data is sent to the Dynatrace cluster via the beacon URL.

> **Note:** Auto-instrumentation covers both `View`-based layouts (XML + Activities/Fragments) and Jetpack Compose. Jetpack Compose auto-instrumentation is enabled by default from Dynatrace Android Gradle plugin 8.271. See the next section for when manual actions still help.

5. Jetpack Compose Integration

Jetpack Compose is auto-instrumented. From Dynatrace Android Gradle plugin 8.271, the plugin instruments Compose interactions by default -- `Modifier.clickable`, `Modifier.combinedClickable`, `Modifier.toggleable`, `Modifier.swipeable`, `Modifier.pullRefresh`, `Slider` / `RangeSlider` and `HorizontalPager` -- so ordinary taps, toggles and swipes are captured without code. Compose auto-instrumentation for Session Replay is on by default from plugin 8.325.

> **Correction (09/28/2026).** Earlier revisions said Compose taps and navigation could not be detected automatically and told readers to wrap every `onClick` in `enterAction` / `leaveAction`. On plugin 8.271+ that duplicates the automatically captured actions. Use manual actions only for the business-level pattern below.
>
> **Sources:** Instrument Android apps (DT docs) — *"Jetpack Compose auto-instrumentation is enabled by default starting with Dynatrace Android Gradle plugin version 8.271."*, Configure monitoring capabilities (DT docs).

Business-Level Actions Spanning Several Interactions

Manual actions are still the right tool when one **business step** spans several interactions or asynchronous work -- a checkout, a search that fans out, a multi-screen form. Use `Dynatrace.enterAction()` and `action.leaveAction()` around the whole step:

```
import com.dynatrace.android.agent.Dynatrace

@Composable
fun ProductListScreen() {
    LazyColumn {
        items(products) { product -&gt;
            ProductCard(
                product = product,
                onClick = {
                    // Manual action for Compose interactions
                    val action = Dynatrace.enterAction("Tap on ${product.name}")
                    navigateToDetail(product.id)
                    action.leaveAction()
                }
            )
        }
    }
}
```

When a Manual Action Adds Value

Pattern	Example	Why manual
Multi-step flow	`enterAction("Checkout flow")`	One action for a flow that spans screens
Async business step	`enterAction("Search: $query")`	Duration should include the coroutine, not just the tap
Named business outcome	`enterAction("Submit Order")`	A stable business name for dashboards, independent of UI labels

Nested Actions with Network Calls

If an action triggers a network request, the Dynatrace agent automatically correlates the HTTP call as a child of the open action:

```
@Composable
fun CheckoutButton(cartId: String) {
    Button(onClick = {
        val action = Dynatrace.enterAction("Tap Checkout")
        viewModel.submitOrder(cartId) // OkHttp call auto-linked
        action.leaveAction()
    }) {
        Text("Checkout")
    }
}
```

Coroutine-Safe Pattern

When using Kotlin coroutines, ensure `leaveAction()` is called after the async work completes:

```
fun onSearchClicked(query: String) {
    val action = Dynatrace.enterAction("Search: $query")
    viewModelScope.launch {
        try {
            val results = repository.search(query)
            _state.value = SearchState.Success(results)
        } catch (e: Exception) {
            action.reportError("Search failed", e)
            _state.value = SearchState.Error(e.message)
        } finally {
            action.leaveAction()
        }
    }
}
```

> **Tip:** Always call `leaveAction()` in a `finally` block to avoid orphaned actions that can skew session data.

6. ProGuard & R8 Configuration

With R8 (the default), you add nothing. OneAgent for Android and its transitive dependencies ship their own keep rules for R8, and R8 applies them. Two cases need attention:

Situation	What to do
A **third-party obfuscator** instead of R8	Configure it to honor OneAgent's keep rules — otherwise you can get runtime errors from incorrectly obfuscated classes
AGP features that **filter dependency keep rules** (for example `ignoreFrom`)	Make sure OneAgent's rules are not filtered out

A blanket `-keep class com.dynatrace.** { *; }` is not required with R8; add it only if a third-party tool cannot read the shipped rules.

Mapping Files for Crash Deobfuscation

Readable Android stack traces need the build's `mapping.txt` in Dynatrace. Upload it for **every released version** — through the UI, or from CI with the Mobile Symbolication API:

```
# token scope: DssFileManagement
curl -X PUT "https://{your-environment-
id}.live.dynatrace.com/api/config/v1/symfiles/{applicationId}/{packageName}/ANDROID/{versionCode}/{versionName}" \
  -H "Authorization: Api-Token {token}" \
  -H "Content-Type: text/plain" \
  --data-binary @app/build/outputs/mapping/release/mapping.txt
```

The upload limit is 100 MiB per file (500 MiB uncompressed). A crash from a version with no mapping file stays obfuscated, so make the upload part of the release pipeline rather than a manual step. MOBL-06 covers crash analysis.

> **Sources:** Support and limitations — Android (DT docs) — *"OneAgent for Android and its transitive dependencies provide ProGuard rules designed for R8. If you use third-party obfuscation tools instead of R8, you are responsible for configuring them to honor the required keep rules."*; Mobile Symbolication API — PUT upload file (DT docs) — *"Uploads a symbol file (Android mapping file and iOS/tvOS symbol extract file) for the specified version of a mobile app."*, *"you need an access token with DssFileManagement scope"*.

7. Verifying Data in Dynatrace

After building and running your instrumented Android app, use the following DQL queries to verify that monitoring data is arriving in Dynatrace.

```
// Find Android mobile applications
// PREFERRED -- Smartscape. FRONTEND carries both mobile and web apps, so the
// frontend.type filter is what narrows this to mobile.
smartscapeNodes "FRONTEND"
| filter frontend.type == "mobile"
| filter contains(name, "Android", caseSensitive: false)
    or contains(toString(tags), "Android", caseSensitive: false)
| fields name, id, id_classic, tags
| sort name asc

// CORRECTION (07/30/2026): a previous revision claimed dt.entity.mobile_application had no
// Grail Smartscape equivalent. It does -- FRONTEND filtered on frontend.type == "mobile"
// (dt.smartscape.frontend). SaaS 1.344 (07/27/2026, staged tenant rollout from 07/29/2026)
// makes Smartscape the primary surface for the Digital Experience apps; verify it has
// reached your tenant. The classic table below still works and remains a real fallback.
// FALLBACK (classic surface -- still functional):
// fetch dt.entity.mobile_application
// | filter contains(toString(entity.name), "Android") or contains(toString(tags), "Android")
// | fields entity.name, id, tags
// | sort entity.name asc
```

The query above searches for mobile application entities with "Android" in their name or tags. If your app appears in the results, the Dynatrace environment is aware of it.

Next, check whether the app is sending user action data:

```
// Recent Android user actions (last hour)
fetch user.events, from:-1h
| filter dt.rum.application.type == "mobile" and os.name == "Android"
| filter characteristics.has_user_action
| summarize action_count = count(), by:{frontend.name, ui_element.detected_name, interaction.type}
| sort action_count desc
| limit 20
```

This query shows the most frequent user actions reported from Android devices in the last hour. Look for:

- **Load actions** from Activity/Fragment lifecycle (auto-instrumented)
- **Tap actions** from button clicks or Compose `enterAction()` calls
- **Custom actions** from your manual instrumentation

Finally, verify the full range of event types being captured:

```
// Android app event-type inventory -- one count per event characteristic.
// Select event types with the stable characteristics.has_* flags, not characteristics.classifier
// (corrected 11/11/2026): the dictionary calls classifier "not intended for query usage" and
// Semantic Dictionary 1.349 removes it from the user-event models. The flags can overlap
// (a failed request is both a request and an error), so the columns need not add up to total_events.
fetch user.events, from:-1h
| filter dt.rum.application.type == "mobile" and os.name == "Android"
| summarize {total_events = count(),
    user_actions = countIf(characteristics.has_user_action == true),
    requests = countIf(characteristics.has_request == true),
    errors = countIf(characteristics.has_error == true),
    crashes = countIf(characteristics.has_crash == true),
    anrs = countIf(characteristics.has_anr == true),
    app_starts = countIf(characteristics.has_app_start == true)}
```

What the Data Looks Like

Mobile RUM data is stored in the Grail `user.events` and `user.sessions` stores, not in `bizevents`. Each `user.events` record carries `characteristics.*` flags that name its event type -- for example `characteristics.has_user_action`, `has_request`, `has_crash`, `has_anr`, `has_app_start`, and `has_error` (with `characteristics.is_api_reported` for errors reported through the SDK). The inventory query above counts each flag. The flags can overlap -- a failed request carries both `has_request` and `has_error` -- so the counts need not add up to the total.

> **Correction (11/11/2026).** Earlier revisions grouped by `characteristics.classifier`. The User events dictionary page describes it as *"Used for internal optimization when storing the data and not intended for query usage."*, and **Semantic Dictionary 1.349** removes it from the user-event models. A tenant that has not reached the change still returns the field (on the validation tenant on 11/11/2026 it was populated on every user event), so a classifier query keeps working there and then silently matches nothing once the change arrives. Semantic Dictionary 1.350 re-lists it under the Basic user-event model; the User events page still calls it not intended for query usage, so select with the `has_*` flags.
>
> **Sources:** User events (DT docs) — *"Used for internal optimization when storing the data and not intended for query usage."*; Semantic Dictionary changelog 1.349 (DT docs); Semantic Dictionary changelog 1.350 (DT docs). **Dictionary:** `characteristics.classifier` (`experimental`), `characteristics.has_user_action`, `has_request`, `has_error`, `has_crash`, `has_anr`, `has_app_start` (`stable`), read 11/11/2026.
>
> **Correction (09/28/2026).** Earlier revisions listed `com.dynatrace.mobile.*` event types. Those do not exist; mobile events are typed by the `characteristics.*` flags above (semantic dictionary models `rum_mobile_user_action`, `rum_request`, `rum_crash`, `rum_anr`, `rum_app_start`, `rum_api_reported_error`, read 09/28/2026).

Troubleshooting Checklist

If no data appears:

Check	Action
RUM on the frontend	Frontend created in Experience Vitals; a RUM Classic frontend needs RUM turned on (Section 1) — with it off, `user.events` stays e
Application ID	Verify it matches the Dynatrace mobile app configuration
Beacon URL	Confirm the URL is reachable from the device/emulator
Network access	Ensure the device has internet connectivity
Plugin applied	Verify `com.dynatrace.instrumentation` appears in Gradle sync output
Build variant	Confirm the correct variant config (debug vs. release) is active

Check	Action
userOptIn	If set to `true`, OneAgent collects nothing until the app calls `Dynatrace.applyUserPrivacyOptions(UserPrivacyOptions.builder().withDataCollectionLevel(DataCollectionLevel.US` after the user consents (MOBL-09 §3)

Summary

In this notebook, you learned:

- How to create and configure a mobile application in the Dynatrace UI
- How to apply the Dynatrace Android Gradle plugin in the top-level `build.gradle.kts` and configure variant-specific configurations
- The full range of auto-instrumented interactions (Activities, Fragments, clicks, HTTP requests)
- That Jetpack Compose is auto-instrumented from plugin 8.271, and when a manual business-level action still helps
- That R8 keep rules ship with the agent, and that mapping files must be uploaded for every release
- How to verify Android monitoring data using DQL queries

Next Steps

Next Notebook	Topic
MOBL-04	Cross-Platform Frameworks
MOBL-05	User Action Tracking

References

- Instrument Android apps (DT docs)
- Dynatrace Mobile RUM Overview
- Instrumentation via Dynatrace Android Gradle plugin (DT docs)
- Configure plugin for instrumentation processes (DT docs)
- OneAgent SDK for Android (DT docs)
- Manual instrumentation with OneAgent SDK for Android (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.